

Scalable and Low-Cost NTT Architecture With Conflict-Free Memory Access Scheme

Zhenyang Wu¹, Ruichen Kan, Jianbo Guo², and Hao Xiao¹

Abstract—This brief proposes a *scalable multistage and multipath* architecture for variable number-theoretic transform (NTT). The proposed architecture adopts multiple parallel paths, each of which uses cascaded radix-2 butterfly units (BFUs). The radix-2 scheme simplifies the control logic and the cascaded BFU structure reduces the amount of RAM banks and the frequency of memory accesses. Moreover, a conflict-free and hardware-friendly in-place memory mapping scheme is proposed to ease the adaption to multiple paths, letting it be scalable for various throughputs. Compared with state-of-the-art works, the proposed architecture uses fewer resources and has better area-time product performance without penalty in throughput.

Index Terms—Conflict-free, cryptography, memory access pattern, number-theoretic transform (NTT), scalable.

I. INTRODUCTION

Postquantum cryptography and fully homomorphic encryption are hot topics of modern cryptography. However, both of them are extremely computationally intensive, which is caused by the large number of polynomial multiplications (PMult). NTT/INTT are commonly used to speed up PMult, and thus, the efficient NTT/INTT architecture is essential.

Considering the tradeoff between area and performance, authors [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], and [13] used memory-based NTT structure, which requires complex memory access schemes to avoid memory conflict. To solve this problem, [10], [11], [12] adopt ping-pong storage, but it inevitably results in the storage overhead. Thus, the in-place structure is used in [2], [3], [4], [5], [6], [7], [8], and [9] to save memory resources. In [6], a complex memory mapping between the original address and the actual address is needed, which consumes a lot of logic resources. Similarly, Zhu et al. [14] adopted a nonconflict vector address mapping scheme, but complex interconnect networks are needed between input and output. Mert et al. [3], [9] proposed a method of delaying the writing back of the output of the butterfly unit (BFU) to avoid memory conflict. However, it makes the memory mapping scheme more complex. Moreover, to improve the throughput of memory-based architectures, a lot of designs increase parallelism by adopting multiple BFUs. Therefore, the bandwidth of the data routing network between BFUs and memories becomes the performance bottleneck. To deal with this problem, Cheng et al. [5] proposed a radix-4 architecture, which consists of two layers of BFUs to improve the area efficiency. Zhao et al. [8] proposed a memory mapping scheme for the radix-4 structure, simplifying the memory access pattern and routing network. However, the BFU structure in radix-4 is much more complex and less flexible than radix-2.

Most existing works propose dedicated architectures for specific NTT with a fixed modulus q and polynomial length N . However,

various application scenarios require different security parameters and cost-performance tradeoffs, and, therefore, scalability is also crucial for NTT architectures. However, the greatest challenge for scalable NTT architectures is the efficient dataflow with a simple and conflict-free memory access scheme as the parallelism of BFUs increases. Chen et al. [4] proposed a scalable architecture and demonstrated the radix-4 BFU scheme can reduce the number of memory banks and memory access time compared to radix-2 by reducing the number of interconnect networks. Mu et al. [7] further optimized this architecture by extending the scalability on both the width and depth of the architecture. However, its number of necessary PEs n_{pe} grows exponentially with the depth d_{pe} , where n_{pe} is a multiple of $d_{pe} \times 2^{d_{pe}-1}$. Thus, when the depth scales, the number of PEs grows rapidly, leading to more complex control logic and memory patterns. Liu et al. [13] proposed a scalable NTT architecture that adopts multiple parallel radix-2 BFUs and an out-of-place memory mapping scheme to get rid of conflicts. However, it does not take into account the higher memory access frequency of the out-of-place memory scheme.

Therefore, this brief proposes a *scalable multistage and multipath* NTT architecture supporting variable modulus and polynomial length. Unlike traditional scalable architectures based on high-radix schemes, the proposed architecture adopts multiple parallel paths, each of which consists of cascaded radix-2 BFUs. The radix-2 scheme simplifies the control logic and the cascaded BFU structure reduces the amount of RAM banks required for parallel processing and the frequency of memory accesses. Moreover, a conflict-free and hardware-friendly in-place memory access scheme is proposed, which can be easily scaled to multiple paths. Experimental results show that, compared to state-of-the-art works, this design achieves a good balance between area and performance.

II. NUMBER-THEORETIC TRANSFORM

NTT is a discrete Fourier transform (DFT) defined in the finite field, that is, in our case, $\mathbb{Z}_q[x]/(x^N + 1)$. The input of NTT is a vector $a = \{a_0, a_1, \dots, a_{N-1}\}$, which can be regarded as a sequence of coefficients of the polynomial $a(x) = \sum_{i=0}^{N-1} a_i x^i$, and the output vector $A = \{A_0, A_1, \dots, A_{N-1}\}$ is another representation form of the polynomial in the NTT domain. The calculation formula of the N -point NTT is defined as follows: $A_i = \sum_{j=0}^{N-1} a_j \omega^{ij} \bmod q$, where ω is the N th primitive root of unity and q is the modulus. INTT is similar as NTT, which is defined as $a_i = N^{-1} \sum_{j=0}^{N-1} A_j \omega^{-ij} \bmod q$.

In polynomial multiplication, negative wrapped convolution is commonly used to eliminate polynomial doubling and reduction operations. Let ω and φ be the N th and $2N$ th primitive roots of unity in $\mathbb{Z}_q$, respectively. A typical NTT-based PMult is described as follows.

Preprocessing: $a'_i = a_i \cdot \varphi^i$ and $b'_i = b_i \cdot \varphi^i$.

Multiplication: $c' = \text{INTT}(\text{NTT}(a') \odot \text{NTT}(b'))$.

Postprocessing: $c_i = c'_i \cdot \varphi^{-i}$.

The pre- and postprocessing operations can be merged into the NTT and INTT operations. Therefore, decimation-in-time (DIT) and decimation-in-frequency (DIF) butterflies are used for the merged

Received 6 June 2024; revised 29 August 2024, 27 October 2024, and 5 December 2024; accepted 2 January 2025. Date of publication 15 January 2025; date of current version 28 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 61974039. (Corresponding author: Hao Xiao.)

The authors are with the School of Microelectronics, Hefei University of Technology, Hefei 230601, China (e-mail: xiaohao@hfut.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2025.3526261

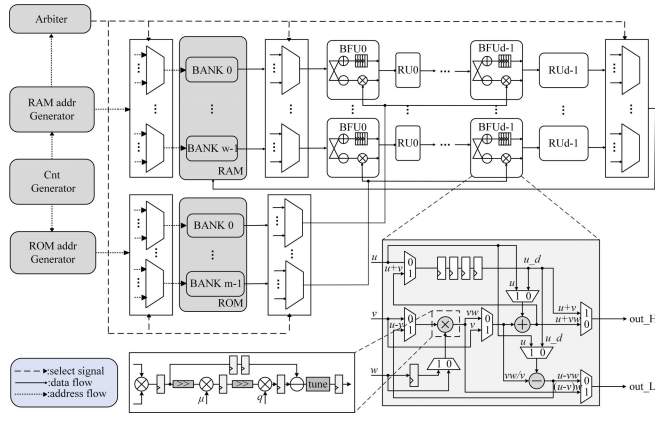


Fig. 1. Proposed architecture for NTT/INTT.

NTT and INTT, and by leveraging the binary property $\varphi_{2N}^{-i} = -\varphi_{2N}^i$, $\varphi_{2N}^{-i} \bmod q = -\varphi_{2N}^i \bmod q$, the twiddle factors of radix-2 NTT can be reused by INTT to save storage.

III. PROPOSED HARDWARE ARCHITECTURE

A. Overall Architecture

Fig. 1 shows the proposed NTT architecture which consists of w separate datapaths, each of which has d cascaded radix-2 BFUs and d reordering units (RUs). This architecture is scalable from both width w and depth d , independently, where d is a factor of $\log_2 N$ and $w \leq N/2$. Each datapath is assigned a RAM bank for storing the temporary data. The ROM is used to store the N twiddle factors, each of which is stored only once for saving area. To support the $w \times d$ BFUs to use the twiddle factors simultaneously, the ROM is separated into m banks, where $m = w \times \lceil \log_2 d \rceil$, with every two BFUs share one bank. The arbitrator controls the $w \times w$ -to-1 MUX array to map the path to the correct memory bank. The BFU, which consists of a modified modular adder [2], a modular subtracter, and a modular multiplier, can be reconfigured in both DIT and DIF modes for NTT and INTT. The RU is used to adjust the step size after the BFU so that the output data can be used directly for the calculation at the next stage.

As shown in Fig. 1, the w paths, which consist of multiple cascaded radix-2 BFUs, work in parallel. In each path, the input of the latter BFU is derived from the output of the former BFU, which saves the bandwidth of input and output, thereby reducing the number of memory banks and the frequency of memory accesses. Due to the reduction of memory banks in each path, the interconnect logic is hardware-friendly when the architecture scales. More importantly, each path in the proposed architecture works independently and the data in different paths do not cross each other during processing, which makes the hardware resources increase linearly when the number of parallel paths scales. Table I compares the memory cost and the memory access time of three architectures. For an N -point NTT, the ping-pong strategy [12] requires 4 RAM banks, the size of which is $2N \log_2 q$, where q is the modulus and the memory access time is $2N$ per iteration (N reads and N writes). The conventional in-place architecture [4], [7] requires 2 RAM banks, the size of which is $N \log_2 q$ and the memory access time is $2N$. However, the proposed architecture stores the two input data for each BFU at the same address in a single RAM bank, and thus, there are only N memory accesses during each iteration, which are $N/2$ reads and $N/2$ writes, respectively.

TABLE I
COMPARISON OF MEMORY ACCESS TIMES

	Ping-pong [12]	In-place [4] [7]	This work
RAM bank	4	2	1
RAM depth	$N/2$	$N/2$	$N/2$
RAM width	$\log_2 q$	$\log_2 q$	$2 \log_2 q$
RAM size	$2N \log_2 q$	$N \log_2 q$	$N \log_2 q$
memory access	$2N$	$2N$	N

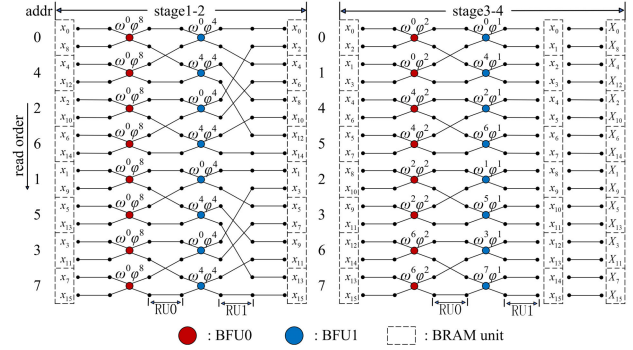


Fig. 2. Proposed dataflow in 16-point NTT.

B. Memory Access Scheme and Dataflow

Due to the step size of NTT changes at each stage, its read and write patterns need to be adapted accordingly, which may result in memory access conflict, especially for in-place architecture. Thus, a conflict-free dataflow scheme is designed for the proposed NTT architecture. Taking a 16-point NTT with 2 cascaded radix-2 BFUs as an example, Fig. 2 illustrates the proposed dataflow. Each pair of BFU inputs are saved in the same memory bank, where points 0–7 are stored in the lower bits of the cell and points 8–15 are stored in the higher bits. After each BFU, an RU is used to adjust the step size, ensuring the output data is correctly mapped to the next stage. It is noted that, as shown in Fig. 2, the output pair of each pipeline path is written back to the RAM in the exact order in which they are read. More details of the RU are explained in Section III-C.

Algorithm 1 describes the detailed memory access scheme of the dataflow. A cycle counter cnt is adopted to trigger a set of sequential numbers that are further bit-reversed to generate the initial read and write-back address addr_ram of the first round iteration. Then, the addr_ram of the next round iteration is generated from the address of the previous round by a circular-right shift. The ROM addresses for accessing twiddle factors are shown in steps 6–10. Since NTT and INTT reuse the same set of twiddle factors, a mode signal is used to switch between NTT and INTT calculation. It is noteworthy that all these RAM and ROM addresses are generated by hardware-friendly operations, such as bit-reverse, shift, and addition, which simplify the hardware implementation.

The proposed memory access scheme can be easily scaled to multiple paths. Fig. 3(a) and (b) shows the address mapping scheme in 2×2 ($w = 2$ and $d = 2$) configuration with $N = 1024$. The addr_ram in Algorithm 1 is denoted as the logical address (logical_addr) and the address mapped to different memory banks is denoted as the physical address (physical_addr). Two RAM banks are necessary for two parallel paths, and thus, there are cnt0 and cnt1 to generate two logical addresses for each path. It is noted that the generation method of logical address in multiple paths remains the same as the one used in a single path. And the highest bit of logical_addr is the Bank_index (BI), and the rest lower bits are the physical_addr of the RAM bank. The input data whose address is 0–

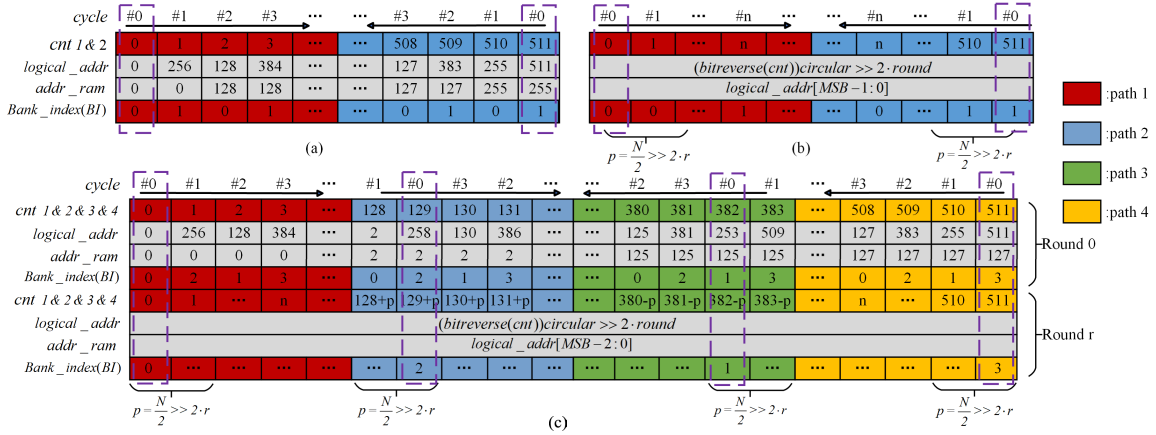


Fig. 3. (a) Pipeline of Round 0 in 2×2 BFUs. (b) Pipeline of Round r in 2×2 BFUs. (c) Pipeline of Round 0 & Round r in 4×2 BFUs.

Algorithm 1 RAM and ROM Address Generation

Input: N denotes N points NTT, cnt denotes cycles, r denotes rounds, d denotes depths, i denotes the code of cascaded BFU (from 0 to d), $mode = 0$ for NTT, 1 for INTT

Output: $addr_ram$, rom_i

```

1  $s \leftarrow 2^d$ ;
2 for  $r = 0$  to  $\lceil \log_s N \rceil - 1$  do
3    $r_{inv} \leftarrow \lceil \log_s N \rceil - r$ ;
4    $t \leftarrow circular\_right(r \cdot d) (BitRverse(cnt))$ ;
5    $addr\_ram \leftarrow t$ ;
6   if  $mode = 0$  then
7      $rom\_i \leftarrow (t \ll r_{inv} \cdot d + i + 1) + (2)^{r_{inv} \cdot d + i}$ ;
8   else
9      $rom\_i \leftarrow ((t \ll r \cdot d + i + 1) + (2)^{r \cdot d + i}) + 1$ ;
10  end
11 end
12 return  $addr\_ram$ ,  $rom\_i$ 
```

Algorithm 2 Cnt Generation in Different Paths

Input: N denotes N points NTT, c denotes cycles, r denotes rounds, w denotes paths, d denotes depths, p denotes offsets

Output: $cnt1$, $cnt2$, $cnt3$, $cnt4$

```

1  $s \leftarrow 2^d$ ;
2 for  $r = 0$  to  $\lceil \log_s N \rceil - 1$  do
3   if  $r \leq 0$  then
4      $p \leftarrow 0$ ;
5   else
6      $p \leftarrow \frac{N}{2} \gg 2r$ ;
7   end
8    $a \leftarrow 1$ ;
9   for  $c = 0$  to  $\frac{N}{2w} - 1$  do
10     $cnt1 \leftarrow c$ ,  $cnt4 \leftarrow N/2 - 1 - cnt1$ ,  $cnt2 \leftarrow$ 
        $N/2w + c + a + p$ ,  $cnt3 \leftarrow 2N/w - cnt2$ ;
11     $a \leftarrow -a$ ;
12  end
13 end
14 return  $cnt1$ ,  $cnt2$ ,  $cnt3$ ,  $cnt4$ 
```

Algorithm 3 ROM Address Mapping Scheme

Input: t denotes ROM addresses, w denotes paths, d denotes depths

Output: ROM_BI , ROM_addr

```

1 if  $w \leq 2$  then
2    $ROM\_BI \leftarrow t[MSB : MSB - \lceil \log_2 d \rceil + 1]$ ;
3    $ROM\_addr \leftarrow t[MSB - \lceil \log_2 d \rceil : 0]$ ;
4 else
5    $ROM\_BI \leftarrow t\{[MSB :$ 
        $MSB - \lceil \log_2 d \rceil + 1], [\lceil \log_2 w \rceil - 2 : 0]\}$ ;
6    $ROM\_addr \leftarrow t[MSB - \lceil \log_2 d \rceil : \lceil \log_2 w \rceil - 1]$ ;
7 end
8 return  $ROM\_BI$ ,  $ROM\_addr$ 
```

four parallel paths, it employs four cnt to generate four $logical_addr$. The detailed cnt generation pattern is formulated by Algorithm 2, which only uses hardware-friendly operations such as addition and shift (division can be implemented by shift). Fig. 3(c) shows a RAM read and write-back address assignment example of using 4×2 BFUs.

As mentioned above, the twiddle factors are stored in m ROM banks, where $m = w \times \lceil \log_2 d \rceil$. Thus, the generated ROM address in Algorithm 1 needs to be mapped to the exact bank. The detailed ROM address mapping scheme is described by Algorithm 3, where only hardware-friendly concat operations are necessary. It is noted that with the increased number of cascaded BFUs and the parallel datapath, the number of ROM banks needs to be increased accordingly.

In summary, the proposed dataflow achieves in-place memory access without any memory conflict. More importantly, the proposed addressing algorithm is path-independent, and thus, scalable and hardware-friendly, which simplifies the hardware implementation.

C. Reordering Unit

According to the dataflow of NTT, the output data of each BFU needs to be reordered before forwarding to the next stage. Due to the straightforward nature of pipeline-based architectures (e.g., SDF and MDC), the distributed memory in each stage can be utilized to delay and reorder the output data simply. The memory-based architecture, on the other hand, requires dedicated RU to alleviate the complexity of the memory control unit. Thus, the proposed architecture uses the RU to adjust the data order of each BFU. As shown in Fig. 4, RU0 and RU1 adopt five and nine registers for reordering and buffering, respectively. Reg4 and reg8 are used as temporary registers to store the data that will be overwritten before output. For RU0, the

255 originally are now located in 0–255 of RAM bank 0 and 256–511 are now located in 0–255 of RAM bank 1. In the scenario of scaling to

TABLE II
IMPLEMENTATION RESULT

Design	Platform	$N, \lceil \log_2 q \rceil$	PE Array	Freq (MHz)	Latency (μs)	Cycles	LUT/ATP	FF/ATP	DSP/ATP	BRAM/ATP
[4]	Virtex-7	1024, 14	2	270	9.5	2574	863/8.2	561/5.3	6/57	2.5/23.75
			4	263	4.9	1294	2.0k/9.9	1.1k/5.2	12/58.8	5/24.5
			8	227	2.9	654	6.3k/18.3	2.1k/6.2	27/78.3	10/29.0
			2×2	270	4.8	1308	1.2k/5.7	969/4.6	12/57.6	3/14.4
			4×2	250	2.7	668	2.9k/8.0	1.9k/5.1	27/72.9	5.5/14.85
[13]	Virtex-7	1024, 14	2	322	8.0	2569	924/7.4	759/6.1	6/47.9	5/39.9
			4	310	4.2	1289	1.6k/6.5	1.3k/5.6	12/49.9	10/41.6
			8	307	2.1	649	2.9k/6.2	2.6k/5.4	24/50.7	20/42.3
[13]	Virtex-7	4096, 24	1	242	101.6	24585	924/93.6	804/81.7	7/711.1	9/914.3
			8	220	13.5	3081	5.5k/76.9	4.9k/67.9	56/784.3	24/336.1
[13]	Virtex-7	4096, 60	1	154	159.6	24585	1.9k/304.6	1.8k/293.6	42/6.7k	17/2.7k
			8	150	20.5	3081	14.1k/290.0	12.5k/257.3	336/6.9k	41/842.1
[7]	Virtex-7	1024, 14	2×2	189	6.9	1294	1.5k/10.1	930/6.4	13/89.7	4.5/31.1
			4×2	179	3.7	654	3.5k/12.8	1.8k/6.6	25/92.5	7.5/27.8
[7]	Virtex-7	4096, 24	4×2	157	19.7	3079	5.7k/111.6	3.2k/62.8	33/650.1	8.5/167.5
			8×2	121	12.8	1543	14.6k/186.8	6.5k/82.8	80/1024.0	12/153.6
[7]	Virtex-7	256, 13	2×2	278	1.0	270	1288/1.3	888/0.9	12/12.0	4.5/4.5
			1×2	310	8.3	2580	465/3.8	458/3.8	6/49.8	1/8.3
Proposed	Virtex-7	1024, 14	2×2	299	4.3	1300	926/4.0	943/4.1	12/51.6	2/8.6
			4×2	283	2.3	660	1.9k/4.3	1.9k/4.5	24/55.2	4/9.2
Proposed	Virtex-7	256, 13	1×2	314	1.7	532	431/0.7	449/0.8	6/10.2	1/1.7
			2×2	304	0.9	276	796/0.7	831/0.8	12/10.8	2/1.8
Proposed	Virtex-7	4096, 24	2×2	235	26.3	6179	2.2k/57.8	2.4k/63.1	24/636.0	6/157.8
			4×2	214	14.5	3107	4.2k/60.9	4.6k/66.7	48/696.0	8/116.0
Proposed	Virtex-7	4096, 60	2×2	160	38.6	6179	5.6k/216.16	6.7k/258.6	168/6.5k	14/540.4
			4×2	150	20.7	3107	10.9k/225.63	12.9k/267.0	336/7.0k	16/331.2

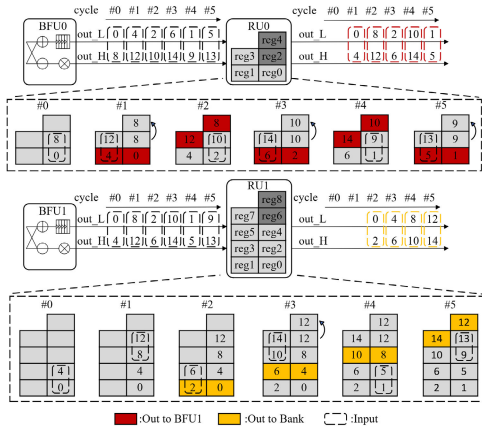


Fig. 4. RU in path 1.

outputs of BFU0, out_L and out_H, are written into reg0 and reg2, respectively, in cycle #0. In cycle #1, the next pair of outputs are written into reg1 and reg3, respectively, and meanwhile, the data in reg2 is copied to reg4 to avoid being overwritten in the following cycle. At this time, a 1-bit counter starts to work for selecting the output of RU0 {reg1, reg0} and {reg3, reg4}. For example, if the output of BFU0 is [(0, 8) (4, 12) (2, 10) (6, 14)], then the output after RU0 is [(0, 4) (8, 12) (2, 6) (10, 14)]. RU1 works similarly, but it takes 4 cycles as a group. RU0 and RU1 introduce 2 and 3 additional cycles in the pipeline.

When scaling to multiple paths, RUs in different paths work similarly. As shown in Fig. 5, the cnt patterns vary in different paths

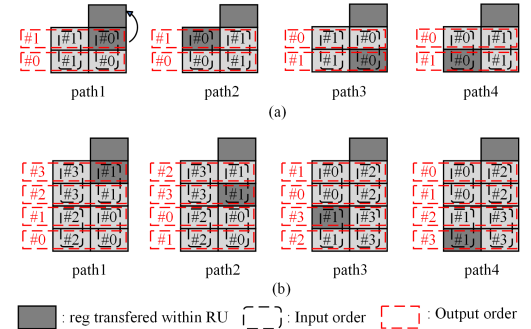


Fig. 5. (a) RU0 in different paths. (b) RU1 in different paths.

and the data order used for BFUs is different. However, the RUs only need to change the overwritten register and the output order to adapt to these differences. It is noteworthy that the register depth of RUs grows only when the number of cascaded BFUs increases, which is not related to N and w , and thus, RUs are well scalable for multiple parallel BFU schemes.

IV. EXPERIMENTAL RESULTS AND EVALUATION

The proposed NTT architecture has been implemented and verified using Xilinx Virtex-7 FPGA with Vivado 2020.2. The RAMs and twiddle factor ROMs in Fig. 1 are implemented using the dual-port block RAMs (BRAMs) of the FPGA.

Our design is compared with existing implementations in Table II with different parameter settings of $N = 256, 1024, 4096, \lceil \log_2 q \rceil = 13, 14, 24, 60, w = 1, 2, 4, d = 1, 2$. Chen et al. [4] uses multiple

parallel radix-2 BFUs of 2, 4, and 8. The additional LUT resources of increasing parallel BFUs are huge, and the LUTs in 8 BFUs are even $3.4\times$ that of ours. The radix-4 strategy is also adopted in [4] to improve area efficiency. However, the LUTs and BRAMs in the radix-4 scheme are still more than ours due to the complexity of the memory access scheme. Mu et al. [7] achieved its scalability by using PE arrays, which leads to $1.5\times$ increment in LUTs when scaling to multiple paths under the same N , whereas our design consistently maintains $1\times$ increment. Also, when using large N and q , our BRAMs increase by only 33% when scaling to multiple paths, while [7] reaches 41%. The resources in [13] increase linearly when the amount of BFUs increases, which is suitable for the scenario with high parallelism. However, due to its out-of-place strategy, large amounts of storage resources are required, leading to a $5\times$ BRAMs that of ours when N and q are small. When N and q are big, the ATP in LUTs of our design improves by 25%.

V. CONCLUSION

In this brief, we have presented a *scalable multistage and multipath* NTT/INTT architecture. We propose a low-complex memory access scheme that is hardware-friendly to support the configurations of different BFU paths. The control logic and path of the overall architecture are simple and efficient due to the proposed scheme, which reduces the hardware resources. Finally, experimental results show that our proposed NTT/INTT architecture can achieve better ATP performance and lower hardware consumption than other existing works.

REFERENCES

- [1] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, no. 2, p. 328356, 2021.
- [2] M. Li, J. Tian, X. Hu, and Z. Wang, "Reconfigurable and high-efficiency polynomial multiplication accelerator for crystals-kyber," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2540–2551, Aug. 2023.
- [3] A. C. Mert, E. Karabulut, E. Ozturk, E. Savas, M. Becchi, and A. Aysu, "A flexible and scalable NTT hardware : Applications from homomorphically encrypted deep learning to post-quantum cryptography," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, Mar. 2020, pp. 346–351.
- [4] X. Chen, B. Yang, and S. Yin et al., "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 1, p. 94126, Nov. 2021.
- [5] Z. Cheng, B. Zhang, and M. Pedram, "A high-performance, conflict-free memory-access architecture for modular polynomial multiplication," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 2, pp. 492–505, Feb. 2024.
- [6] N. Zhang, B. Yang, and C. Chen et al., "Highly efficient architecture of newhope-nist on fpga using low-complexity NTT/INTT," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 2, p. 4972, 2020.
- [7] J. Mu, Y. Ren, and W. Wang et al., "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof and implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 5, pp. 1504–1517, May 2023.
- [8] Y. Zhao, X. Liu, Y. Hu, and H. Xiao, "Design of an efficient NTT/INTT architecture with low-complex memory mapping scheme," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 400–404, Jan. 2024.
- [9] A. C. Mert, E. Karabulut, E. Öztürk, E. Savas, and A. Aysu, "An extensive study of flexible design methods for the number theoretic transform," *IEEE Trans. Comput.*, vol. 71, no. 11, pp. 2829–2843, Nov. 2022.
- [10] Y. Xing and S. Li, "An efficient implementation of the NewHope key exchange on FPGAs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 3, pp. 866–878, Mar. 2020.
- [11] C. Zhang et al., "Towards efficient hardware implementation of NTT for kyber on FPGAs," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
- [12] X. Feng and S. Li, "Accelerating an FHE integer multiplier using negative wrapped convolution and ping-pong FFT," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 66, no. 1, pp. 121–125, Jan. 2019.
- [13] S.-H. Liu, C.-Y. Kuo, Y.-N. Mo, and T. Su, "An area-efficient, conflict-free, and configurable architecture for accelerating NTT/INTT," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 3, pp. 519–529, Mar. 2024.
- [14] Y. Zhu et al., "RePQC: A 3.4-uJ/Op 48-kOPS post-quantum crypto-processor for multiple-mathematical problems," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 124–140, Jan. 2023.